NAME: ANELE DESTINY CHIGOZIE

ID: BAD/2026/TC-9/0130

QUESTION ONE

a. Although I don't frequently order stuffs from Jumia but I often open the app to check out some cool equipment and check prices of some items and compare the price differences with the one I could get from a physical shop. So, when I tap "Login" assuming my previous login session expired, some backend event happens. First off, the login details are sent across a backend server waiting for these details for authentication. The authentication system or script would securely verify the login information given to it for processing, once the system confirms and verifies the details, it generates a unique authentication token and send it back allowing the frontend to store it and update or render the user's home screen.
b. An example of a real-world analogy for backend is a restaurant. The waiters who process a customer order represent the frontend, the kitchen where foods are cooked represents the backend, and the storing medium for these foods (e.g. refrigerator) and the ingredients used in cooking them represents the database. But however, this analogy breaks down because physically comparing the kitchen, it has space or storage limitation, whereas cloud servers for backend can dynamically scale up or cache data to handle millions of simultaneous users without physical contacts.

QUESTION TWO

a. Calling Node.js a programming language is a common misconception that people have because JavaScript is the actual programming language, while Node.js is a runtime environment that lets JavaScript execute outside the web browser.
   At its core, Node.js relies on Google's V8 engine to compile JavaScript directly into native machine code while wrapping it with system-level bindings for file handling and networking.
b. Before 2009, JavaScript was strictly confined to web browsers. Ryan Dahl solved this limitation by combining the V8 engine with an event loop library (libuv), which completely decoupled JavaScript from the browser. This innovation unlocked the ability to use JavaScript for server-side development, file system operations, and building high-performance backend applications.

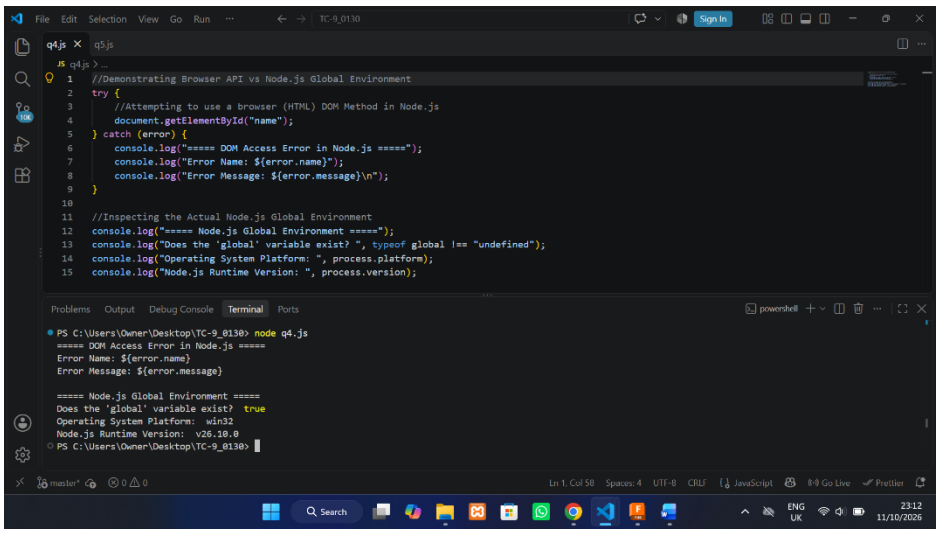
QUESTION THREE

a. Using the real-life scenario of a restaurant, we assume a single fast-food counter worker managing a long line of customers who want to buy food. The physical line of the customers waiting acts as the Event Queue, and the restaurant worker continuously checking the front of the line represents the Event Loop. When an order from a customer requires slow preparation, the worker offloads the task to the kitchen assistants at eh back which functions as a Thread Pool. Once the food is ready, a buzzer notified the restaurant worker, acting just like a Callback.
b. However, the system is going to fail catastrophically if the single restaurant worker is forced to stop and perform a very heavy and time-consuming task like chopping a chunk of vegetable (Ugu) leaves. That blocks the worker entirely, bringing the entire line of the

customers waiting to a standstill. In Node.js server, this is going to happen if a CPU-intensive synchronous task blocks the single threaded Event Loop, freezing the server and causing all incoming requests to time out.

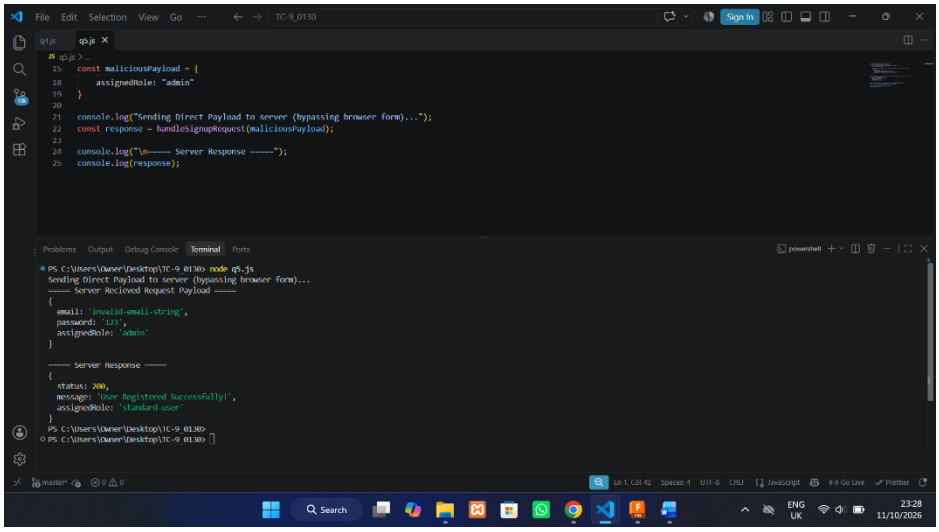
QUESTION FOUR

a. Trying to run “document.getElementById(‘name’)” which is a browser specific code, in Node.js file throws a reference error because the DOM (Document Object Model) and browser API does not exist in a server environment. Node.js runs directly on the operating system without a Graphical User Interface (GUI). Instead of browser’s window object, Node.js uses its own top-level global context simply called ‘global’.



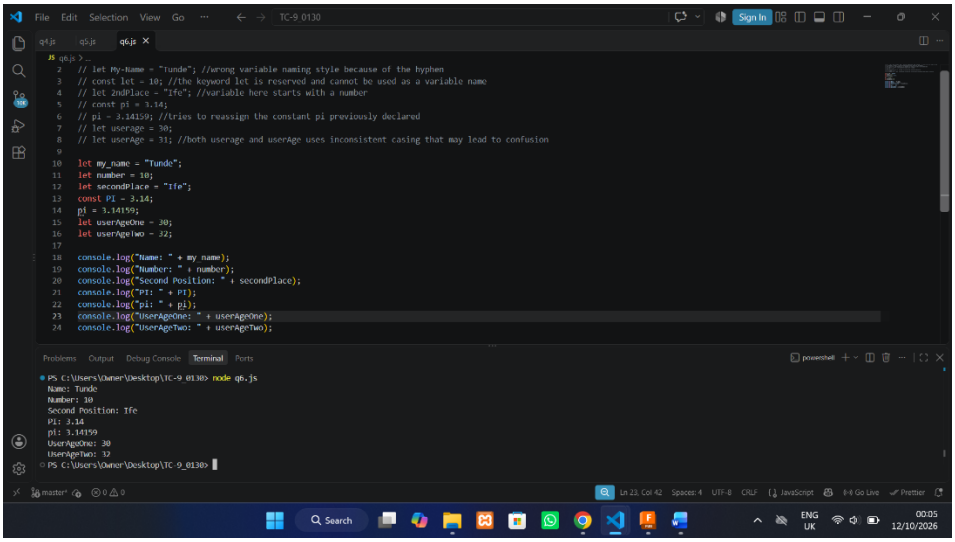
QUESTION FIVE

a. “So, the backend doesn’t need to check again” is a very dangerous assumption because an attacker can bypass client-side checks by disabling JavaScript, editing form attributes in their browser developer tools, or bypassing the browser completely. Instead of using form user interfaces, malicious users can use command line tools to send raw unvalidated HTTP requests and malicious payloads directly to the backend server. Because the frontend can always be bypassed, backend developers must never trust incoming client data and must always implement rigorous server-side validations.



QUESTION SIX

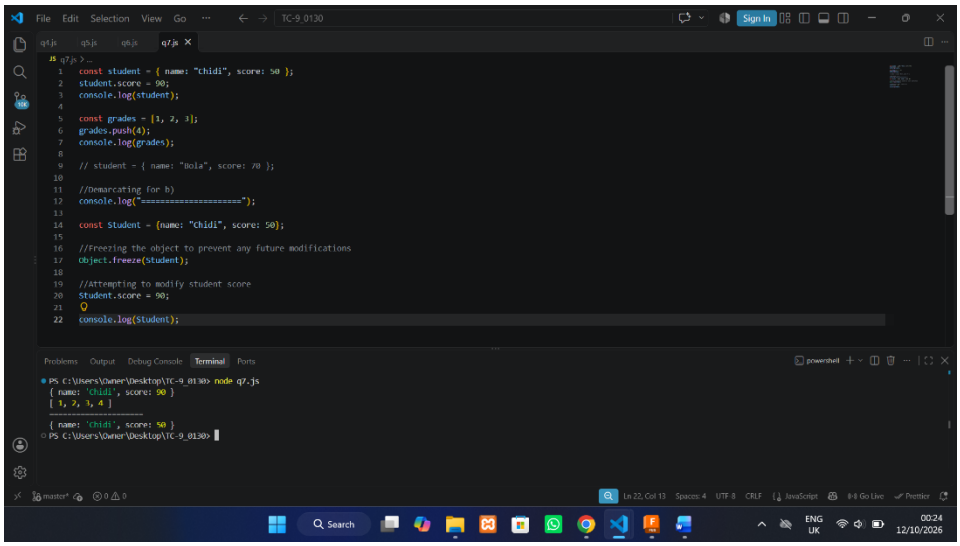
a. let a = 25; *//the variable name a fail to communicate what it holds i.e. if it's an int, float or bool*
let My-Name = "Tunde"; *//wrong variable naming style because of the hyphen*
const let = 10; *// the keyword let is reserved and cannot be used as a variable name*
let 2ndPlace = "Ife"; *//variable here starts with a number*
const pi = 3.14;
pi = 3.14159; *//tries to reassign the constant pi previously declared*
let userage = 30;
let userAge = 31; *//both userage and userAge uses inconsistent casing that may lead to confusion*



b. I would use ***const*** when I want a variable reference to remain completely fixed throughout its lifecycle in my code. I would use ***let*** only when I know for sure that the variables value will need to be updated or reassigned later in my code.
Why you asked us to avoid using ***var***? Because variables declared using ***var*** can leak out of loops and conditional blocks and suffer from hoisting bugs.

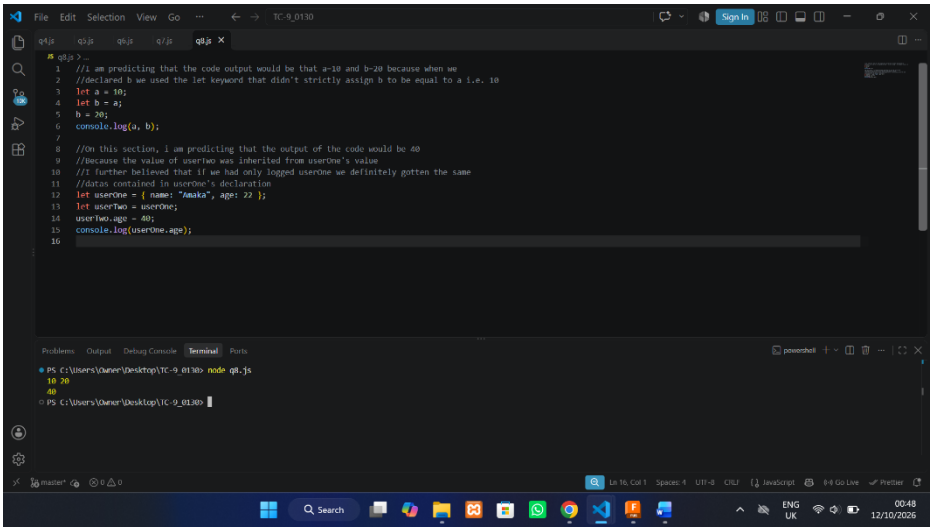
QUESTION SEVEN

a. All lines work except line 9 (student = {name: “Bola”, score: 70};
Because when we declare an object or array with ***const***, we are locking the variable reference so it cannot be reassigned to a brand new object. But however, the contents stored in the memory can still be modified, just as we see in line 2 and line 6 respectively.

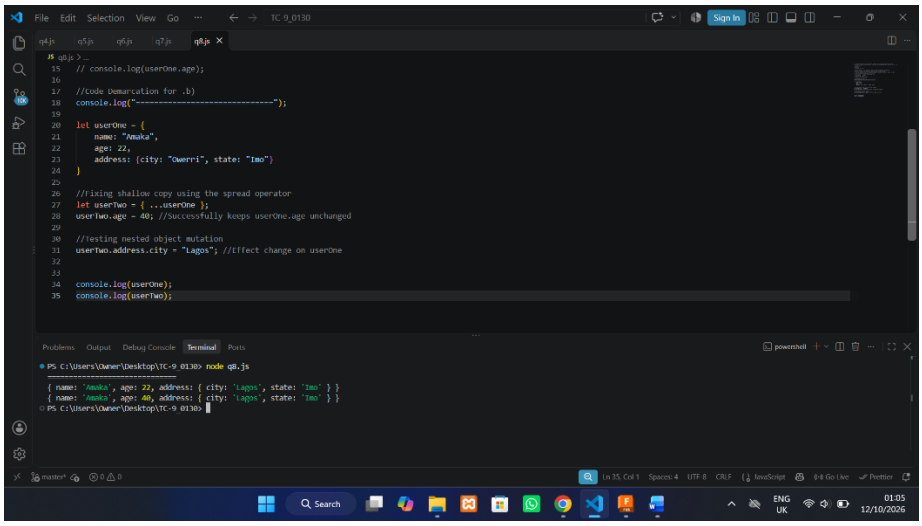


b. Why it fails is because in JavaScript mode, attempting to modify or add properties to a frozen object simply fails quietly without throwing a visible exception. The engine ignores the assignment.

QUESTION EIGHT



a. On Block One, recall that numbers, strings, and Booleans are primitive data type. So when we assign that let ***b = a***, JavaScript creates an independent copy of the value (10) and stores it in a separate memory location. Changing ***b = 20***, has zero effect on ***a*** because they live in completely separate memory slots.
On Block Two, recall that objects and arrays are reference data types. So when we assign ***let userTwo = userOne***; JavaScript does not copy the object, it copies the memory reference address pointing to that object. Both variables point to the same exact object in memory. Therefore, mutating ***userTwo.age = 40***, modifies the shared object causing ***userOne.age*** to output 40.

b. 

I found out that it doesn't work with nested objects. The spread operator only creates a shallow copy. It duplicates top-level primitive properties, but nested objects/arrays (like ***address***) are still copied by reference. Changing a nested property on ***userTwo*** will unintentionally mutate ***userOne*** (In our used example, the address changed).

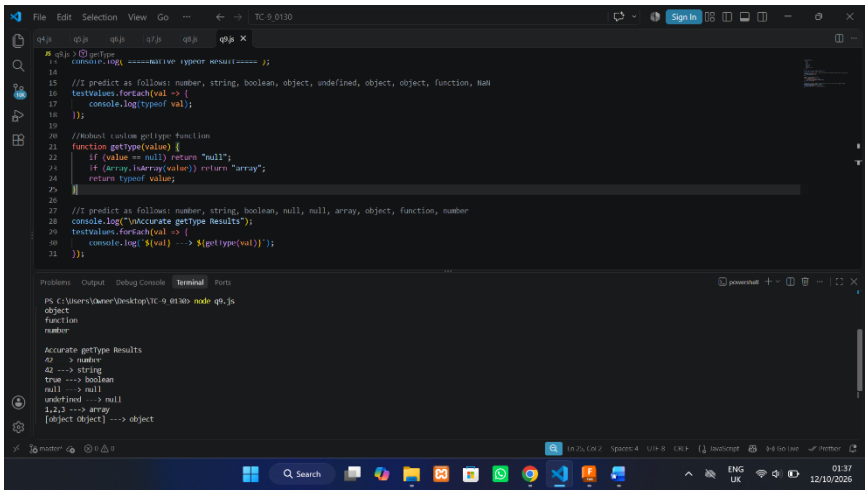
QUESTION NINE

a. I predict as follows: *number, string, boolean, object, undefined, object, object, function, NaN*

b. *typeof null* returns *object.* In a backend application, checking request bodies…., passing *null* passes the check incorrectly, causing runtime errors when trying to read properties of *null.*

Typeof NaN return *number. NaN* which stands for **Not a number**, yet JavaScript classifies its data type as a *number*. This can cause faulty validation checks where invalid numeric fields bypass range or type filters.

c.



QUESTION TEN

